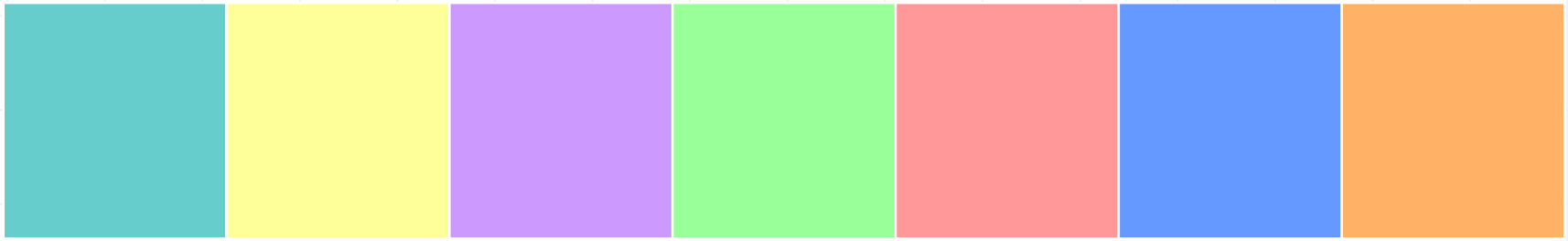
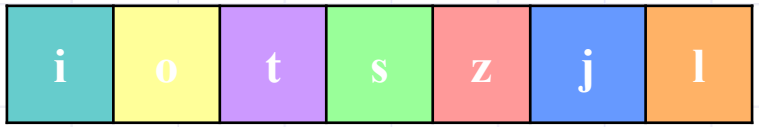


TetrisRL

AI1704 - REL301m Assignment

SE171792 / Nguyen Anh Hao

CONTENTS



01.
TETRIS ENVIROMENT

- Define Piece, Grid, Generators, Rotations, ...
- Define game characteristics

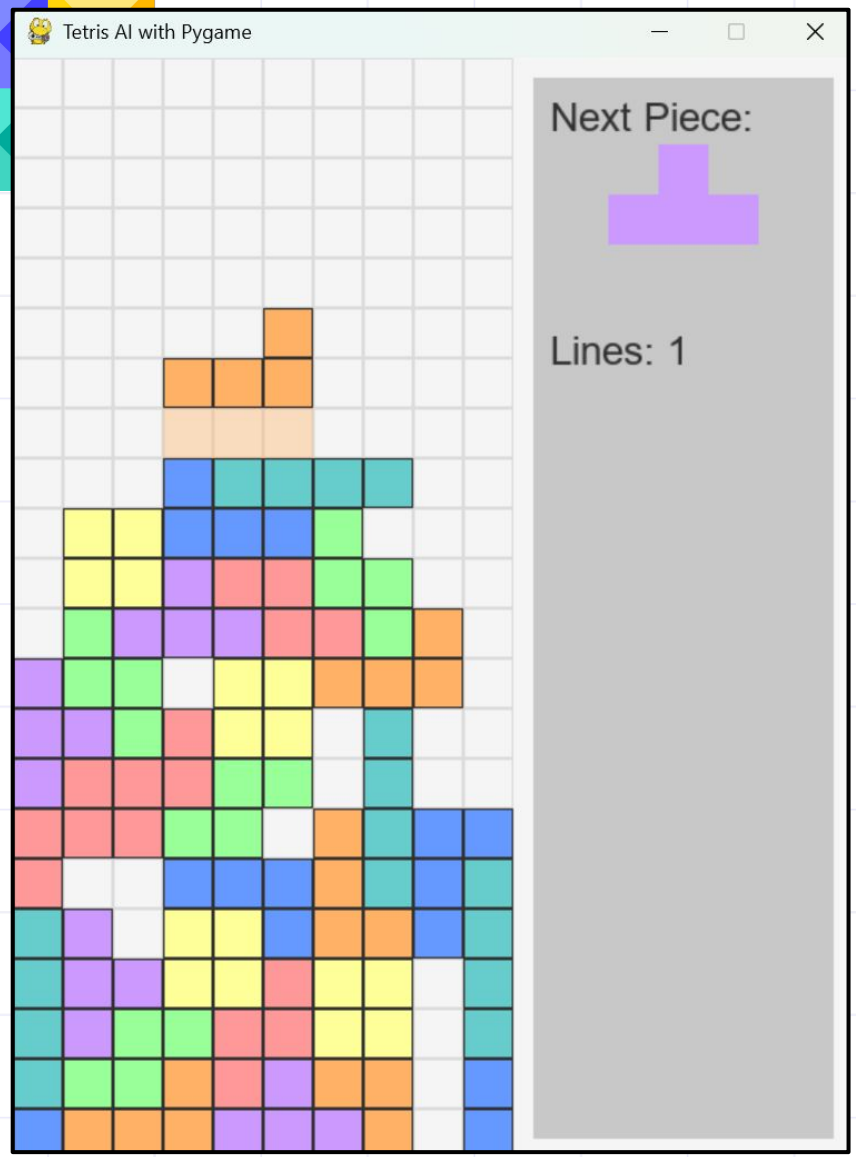
02.
ES ALGORITHM

- The 4 heuristics
- Evolution strategies
- Training strategies

03.
RESULTS & DEMO

- Test results
- Demo agent

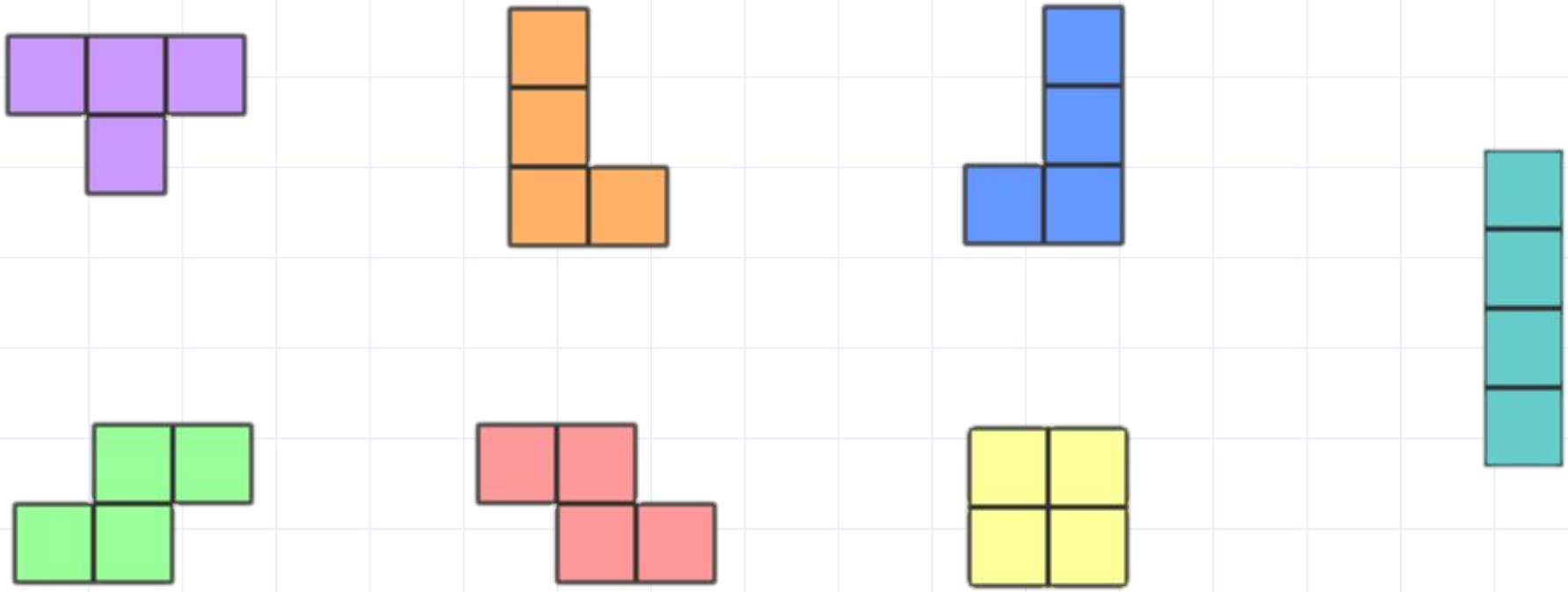
Tetris environment



A basic Tetris environment include:

- Piece
- Grid
- Piece generator
- Rotation method
- Drop method
- Check valid position
- Clear line method

Piece is defined with 7 available shapes:



Piece generator is defined with 2 game mode:

Mode	‘classics’	‘random’
Describe	7-bag random generator: 7 pieces are shuffled and put into a bag, then generator will pop a piece each time. Until no more pieces in bag, the bag will be refilled.	The new piece is randomly selected by random.choice()
Difficulty	Normal	Hard

IDENTIFY TETRIS CHARACTERISTICS

01.

Large action space

Each piece has 1-4 possible rotations, each rotation has 7-10 possible drop place.
-> action space upto >30 .

02.

Delay + sparse reward

Target of Tetris is to clear the rows, which cannot happen immediately after placing a piece.

03.

Randomness

The piece is generated randomly, which mean the agent must be able to adapt unpredictable future state.

04.

Balance challenge

The model must effectively balance between moves that yield quick, small rewards and those that set the stage for larger, future rewards.

Methodology Analysis

Methods	Strengths	Weaknesses
Q-Table	• Simplicity • Deterministic Updates	• Scalability: not suitable for large action space. • Limited Generalization: not suitable for unpredictable future state
DQN	• Scalability • Stabilization Techniques: Incorporates experience replay and target networks to manage delayed rewards and sparse feedback.	• Computational complexity: require careful tuning hyperparameter and more computational resources. • Training instability
ES	• Gradient-free optimization: simplify the optimization process in a complex game as Tetris. • Parallelization: speeding up exploration process by apply multiple policies simultaneously.	• Policy dependent: unreliable policy strongly affect the evolution process. • Balance challenge: balancing exploration and exploitation must be managed carefully.

EVOLUTION STRATEGIES

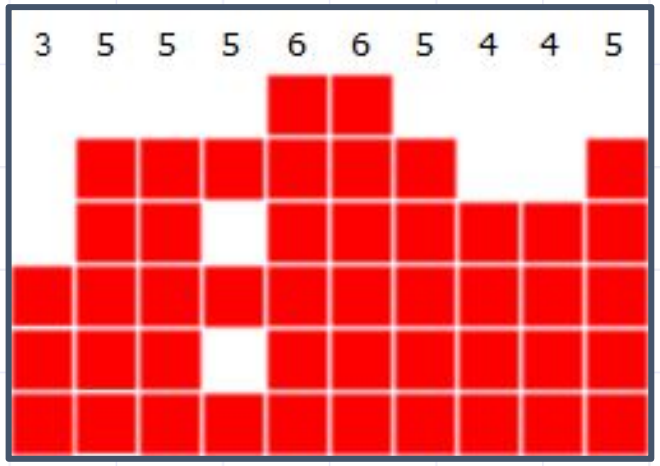
FIND A HEURISTIC FUNCTION FOR POLICY

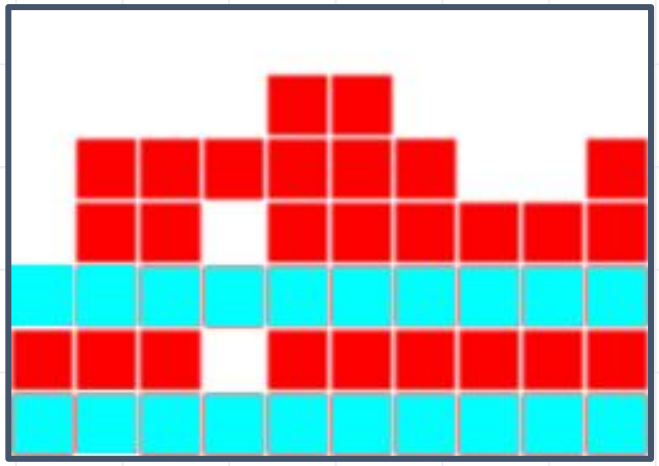
Through research process, we have found a reliable work, thanks to Lee Yiyuan from Rice University, Singapore. Source: CodeMyRoad

HARDWARE COMPATIBLE

ES does not require a powerful GPU as its parallel processing capabilities seem to be better suited to CPUs.

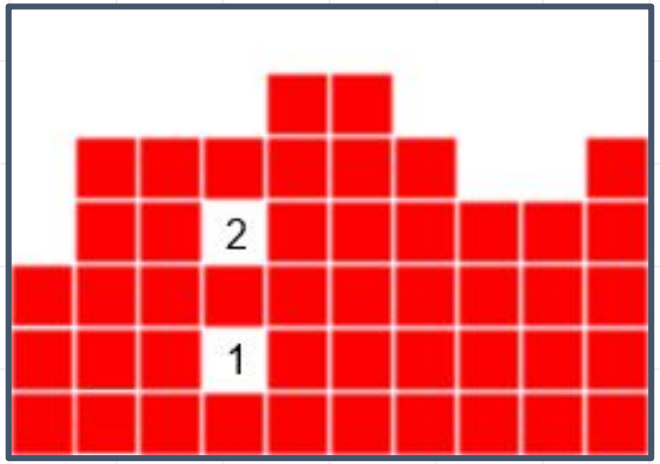
f1:
Aggregate height

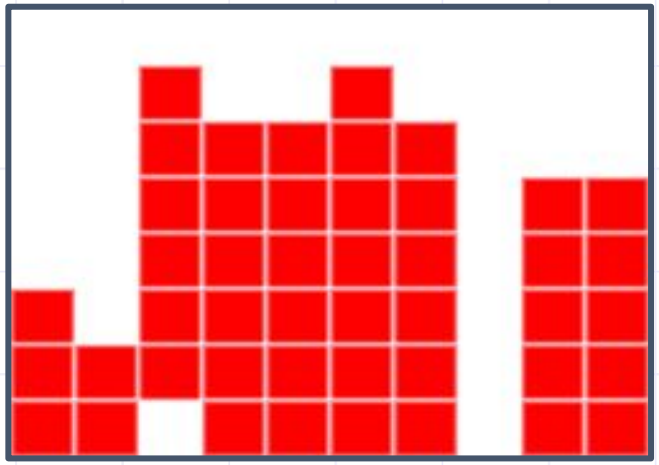




f2:
Clear lines

f3:
Holes



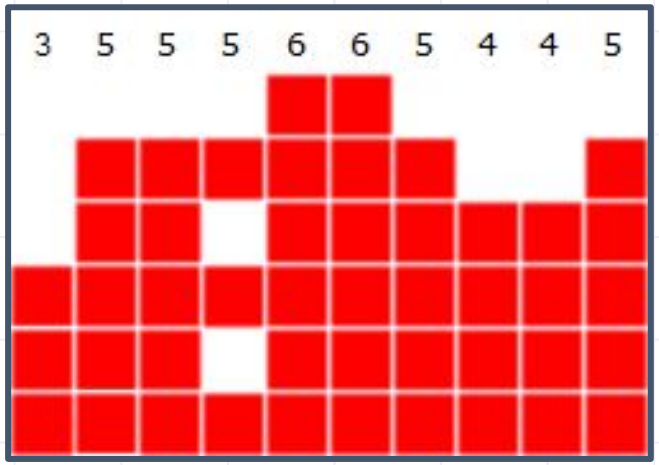


f4:
Bumpiness

THE HEURISTIC FUNCTION

$$\text{reward} = w_1 f_1 + w_2 f_2 + w_3 f_3 + w_4 f_4$$

$$\text{bumpiness} = \sum_{i=1}^{\text{cols}-1} |H[i] - H[i+1]|$$



$$bumpiness = 6 = |3-5| + |5-5| + ... + |4-4| + |4-5|$$

THE HEURISTIC FUNCTION

$$\text{reward} = w_1 f_1 + w_2 f_2 + w_3 f_3 + w_4 f_4$$

THE HEURISTIC FUNCTION

$$\text{reward} = w_1 f_1 + w_2 f_2 + w_3 f_3 + w_4 f_4$$

Evolution Strategies:

1. Agent scan through all possible action in current state and ***choose the action that gain maximum reward(1)*** calculated by the heuristic function.

2. Agent performs action continuously until game_over = True, then ***return the total clear lines*** as reward(2) to update the weights of heuristic function.

Note: reward(1) and reward(2) are different variables.

ES EXPLORATION FLOW

Heuristic weights initialization

Load pre-trained weights to baseline or baseline = [-0.5 0.5 -0.5 -0.5]

Parallel processing

Allow candidates to play <num_games> and record the rewards received

LOAD WEIGHTS → CREATE CANDIDATES → EVALUATE CANDIDATES → UPDATE BASELINE → LOOP FOR N GENERATIONS

Population initialization

Create Gaussian noises and apply to baseline

$$\text{candidates} = \text{baseline} + \sigma \cdot \text{noises}$$

$$\text{standardized_rewards} = \frac{\text{rewards} - \text{mean_reward}}{\text{std}(\text{rewards}) + 10^{-8}}$$

$$\text{update} = \left(\frac{\alpha}{\sigma \cdot \text{population_size}}\right) \cdot (\text{noise}^T \cdot \text{standardized_rewards})$$

$$\text{baseline} \leftarrow \text{baseline} + \text{update}$$

ES EXPLOITATION FLOW

Evaluate training config

Review model performance carefully and config model hyperparameters (sigma, lr, max_moves)

Parallel processing

Allow candidates to play <num_games> and record the rewards received

CONFIGURATION → CREATE CANDIDATES → EVALUATE CANDIDATES → UPDATE BASELINE → LOOP FOR N GENERATIONS

Population initialization

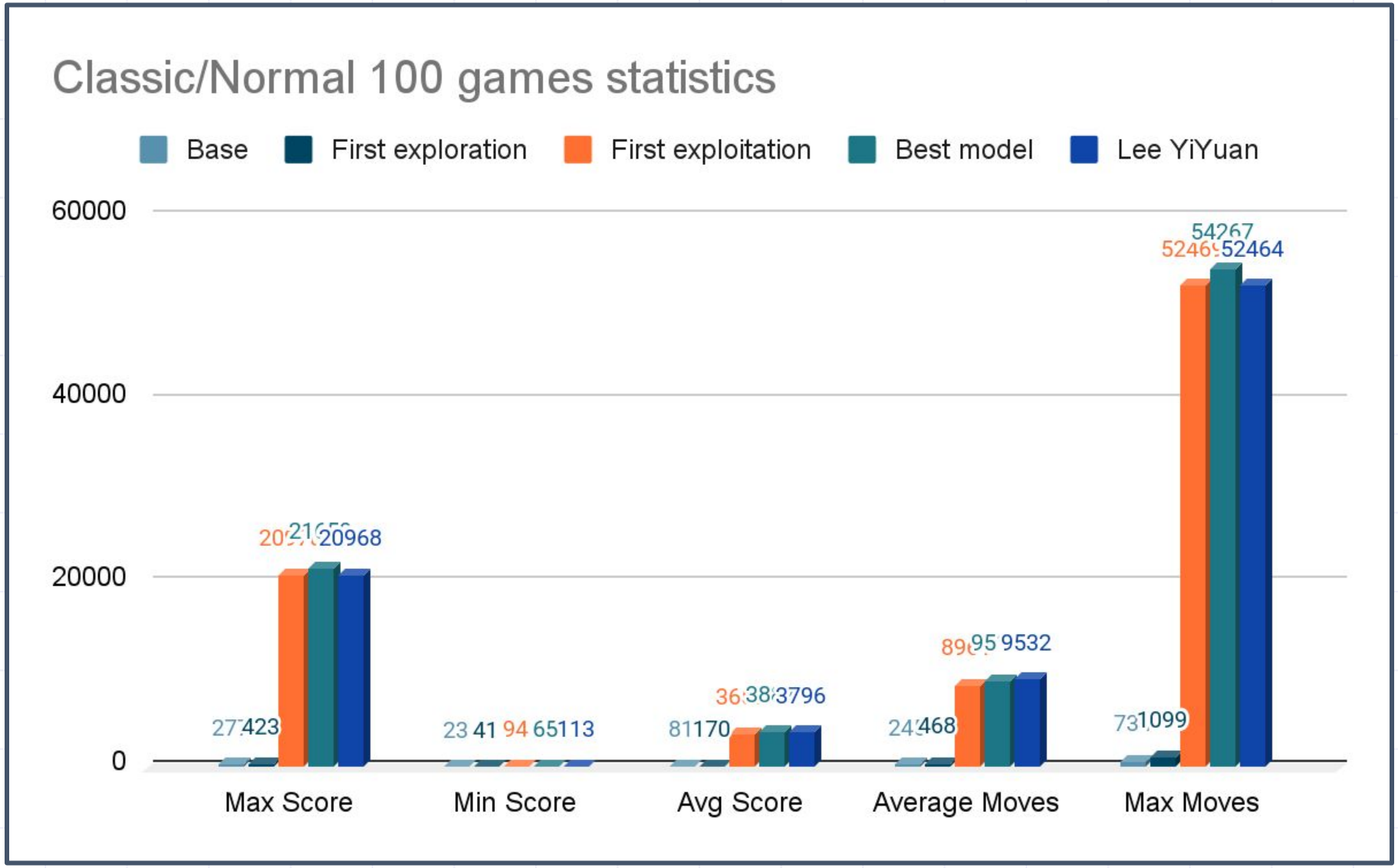
Create a Gaussian noise and apply to baseline

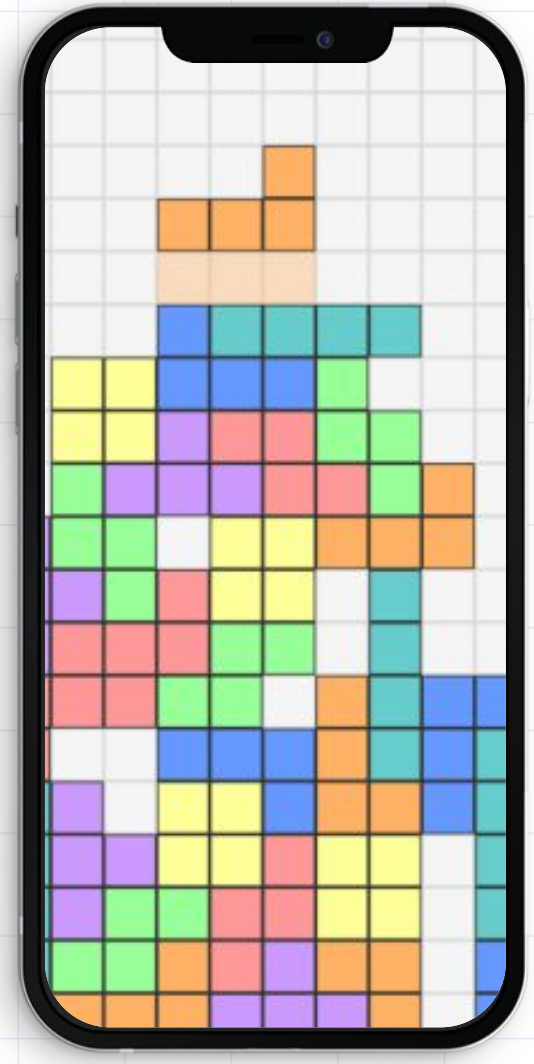
$$\text{candidates} = \text{baseline} + \sigma \cdot \text{noises}$$

Choose elite candidate

Elite candidate is the candidate that perform the best (gain highest rewards)

$$\text{baseline} = \text{elite_candidate}$$


Classic/Normal 100 games statistics
Base
First exploration
First exploitation
Best model
Lee YiYuan
60000
40000
20000
0
20968
423
41
94
65113
81170
3796
468
9532
1099
52464
Max Score
Min Score
Avg Score
Average Moves
Max Moves



AGENT DEMO

THANKS !